Lab 12

PL/pgSQL Named Blocks: Functions and Procedures

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Lab Workflow **2**

2 Database Schema & Data **5**

3 Lab Tasks **8**

1 Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   1. Connect to Your Database:

   Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Verify the Connection:

   Verify that you are connected to the correct database with the following command:

```
\conninfo
```

   Expected output:

```
Database: cse4308_12345, User: student_12345
```

   3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

   Load the SQL file that sets up the tables and inserts initial data.

   1. Load the SQL File:

   Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_12_setup.sql
```

   This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

   After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

   1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

   2. Check Tables:

   Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L12_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

2 Database Schema & Data

Instructions: Save this code as `lab_12_setup.sql` and run it using `\i lab_12_setup.sql` in your `psql` terminal.

```
-- 1. CLEANUP
DROP TABLE IF EXISTS depositor CASCADE;
DROP TABLE IF EXISTS borrower CASCADE;
DROP TABLE IF EXISTS account CASCADE;
DROP TABLE IF EXISTS loan CASCADE;
DROP TABLE IF EXISTS customer CASCADE;
DROP TABLE IF EXISTS branch CASCADE;

-- 2. CREATE TABLES
CREATE TABLE branch
   (branch_name     VARCHAR(15) NOT NULL,
    branch_city     VARCHAR(15) NOT NULL,
    assets          NUMERIC     NOT NULL,
    PRIMARY KEY(branch_name));

CREATE TABLE customer
   (customer_name   VARCHAR(15) NOT NULL,
    customer_street VARCHAR(12) NOT NULL,
    customer_city   VARCHAR(15) NOT NULL,
    PRIMARY KEY(customer_name));

CREATE TABLE account
   (account_number  VARCHAR(15) NOT NULL,
    branch_name     VARCHAR(15) NOT NULL,
    balance         NUMERIC     NOT NULL,
    PRIMARY KEY(account_number),
    FOREIGN KEY(branch_name) REFERENCES branch(branch_name));

CREATE TABLE loan
   (loan_number     VARCHAR(15) NOT NULL,
    branch_name     VARCHAR(15) NOT NULL,
    amount          NUMERIC     NOT NULL,
    PRIMARY KEY(loan_number),
    FOREIGN KEY(branch_name) REFERENCES branch(branch_name));

CREATE TABLE depositor
   (customer_name   VARCHAR(15) NOT NULL,
    account_number  VARCHAR(15) NOT NULL,
    PRIMARY KEY(customer_name, account_number),
    FOREIGN KEY(account_number) REFERENCES account(account_number),
    FOREIGN KEY(customer_name) REFERENCES customer(customer_name));

CREATE TABLE borrower
   (customer_name   VARCHAR(15) NOT NULL,
    loan_number     VARCHAR(15) NOT NULL,
    PRIMARY KEY(customer_name, loan_number),
    FOREIGN KEY(customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY(loan_number) REFERENCES loan(loan_number));

-- 3. POPULATE RELATIONS
INSERT INTO customer VALUES ('Jones',    'Main',       'Harrison');
INSERT INTO customer VALUES ('Smith',    'Main',       'Rye');
INSERT INTO customer VALUES ('Hayes',    'Main',       'Harrison');
```

```
INSERT INTO customer VALUES ('Curry',    'North',      'Rye');
INSERT INTO customer VALUES ('Lindsay',  'Park',       'Pittsfield');
INSERT INTO customer VALUES ('Turner',   'Putnam',     'Stamford');
INSERT INTO customer VALUES ('Williams', 'Nassau',     'Princeton');
INSERT INTO customer VALUES ('Adams',    'Spring',     'Pittsfield');
INSERT INTO customer VALUES ('Johnson',  'Alma',       'Palo Alto');
INSERT INTO customer VALUES ('Glenn',    'Sand Hill',  'Woodside');
INSERT INTO customer VALUES ('Brooks',   'Senator',    'Brooklyn');
INSERT INTO customer VALUES ('Green',    'Walnut',     'Stamford');
INSERT INTO customer VALUES ('Jackson',  'University', 'Salt Lake');
INSERT INTO customer VALUES ('Majeris',  'First',      'Rye');
INSERT INTO customer VALUES ('McBride',  'Safety',     'Rye');

INSERT INTO branch VALUES ('Downtown',   'Brooklyn',    900000);
INSERT INTO branch VALUES ('Redwood',    'Palo Alto',  2100000);
INSERT INTO branch VALUES ('Perryridge', 'Horseneck',  1700000);
INSERT INTO branch VALUES ('Mianus',     'Horseneck',   400200);
INSERT INTO branch VALUES ('Round Hill', 'Horseneck',  8000000);
INSERT INTO branch VALUES ('Pownal',     'Bennington',  400000);
INSERT INTO branch VALUES ('North Town', 'Rye',        3700000);
INSERT INTO branch VALUES ('Brighton',   'Brooklyn',   7000000);
INSERT INTO branch VALUES ('Central',    'Rye',         400280);

INSERT INTO account VALUES ('A-101', 'Downtown',   500);
INSERT INTO account VALUES ('A-215', 'Mianus',     700);
INSERT INTO account VALUES ('A-102', 'Perryridge', 400);
INSERT INTO account VALUES ('A-305', 'Round Hill', 350);
INSERT INTO account VALUES ('A-201', 'Perryridge', 900);
INSERT INTO account VALUES ('A-222', 'Redwood',    700);
INSERT INTO account VALUES ('A-217', 'Brighton',   750);
INSERT INTO account VALUES ('A-333', 'Central',    850);
INSERT INTO account VALUES ('A-444', 'North Town', 625);

INSERT INTO depositor VALUES ('Johnson', 'A-101');
INSERT INTO depositor VALUES ('Smith',   'A-215');
INSERT INTO depositor VALUES ('Hayes',   'A-102');
INSERT INTO depositor VALUES ('Hayes',   'A-101');
INSERT INTO depositor VALUES ('Turner',  'A-305');
INSERT INTO depositor VALUES ('Johnson', 'A-201');
INSERT INTO depositor VALUES ('Jones',   'A-217');
INSERT INTO depositor VALUES ('Lindsay', 'A-222');
INSERT INTO depositor VALUES ('Majeris', 'A-333');
INSERT INTO depositor VALUES ('Smith',   'A-444');

INSERT INTO loan VALUES ('L-17', 'Downtown',   1000);
INSERT INTO loan VALUES ('L-23', 'Redwood',    2000);
INSERT INTO loan VALUES ('L-15', 'Perryridge', 1500);
INSERT INTO loan VALUES ('L-14', 'Downtown',   1500);
INSERT INTO loan VALUES ('L-93', 'Mianus',      500);
INSERT INTO loan VALUES ('L-11', 'Round Hill',  900);
INSERT INTO loan VALUES ('L-16', 'Perryridge', 1300);
INSERT INTO loan VALUES ('L-20', 'North Town', 7500);
INSERT INTO loan VALUES ('L-21', 'Central',     570);

INSERT INTO borrower VALUES ('Jones',    'L-17');
INSERT INTO borrower VALUES ('Smith',    'L-23');
INSERT INTO borrower VALUES ('Hayes',    'L-15');
INSERT INTO borrower VALUES ('Jackson',  'L-14');
```

```
INSERT INTO borrower VALUES ('Curry',    'L-93');
INSERT INTO borrower VALUES ('Smith',    'L-11');
INSERT INTO borrower VALUES ('Williams', 'L-17');
INSERT INTO borrower VALUES ('Adams',    'L-16');
INSERT INTO borrower VALUES ('McBride',  'L-20');
INSERT INTO borrower VALUES ('Smith',    'L-21');
```

Listing 1: lab_12_setup.sql

ewpage

3 Lab Tasks

1. Warm-up
   (a) Print your name and student ID.
   (b) Take two numbers as input and print their product.
   (c) Take a number as input and print whether it is a whole number or a fraction.
2. Execute the provided `lab_12_setup.sql` file and consider the schema for the banking database. Write a procedure to find the top N branches with the highest total loan amounts and list of their borrowers' names (total loan amount means the sum of all loans issued by the branch). The procedure will take N as input and print the details up to N branches. If N is greater than the number of branches, then it will print an error message.
3. Write a procedure to find eligible customers for a special financial award. A customer is eligible for the award if their total deposits are above the average deposit amount and they have at least one loan with an amount greater than 1,000. The procedure will print out the names of the eligible customers.
4. Develop a function to facilitate loan repayment for a customer. The function should accept two input parameters: the customer's name and the payment amount. It should return the customer's remaining loan balance after the payment is applied. The function must ensure that the loan table is updated accordingly. In case the customer has no outstanding loans, the function should display an appropriate error message. Additionally, if the payment amount exceeds the outstanding loan balance, the function must also display an error message.